

FORMULATING FIRST SPECIES COUNTERPOINT WITH INTEGER PROGRAMMING

Tsubasa Tanaka

Tokyo University of the Arts

tanaka.tsubasa@ms.geidai.ac.jp

ABSTRACT

Counterpoint is a theory for composing western polyphonic music. It consists of many rules that regulate melodic and harmonic relationships within each voice or between voices. Because satisfying all of the rules is difficult, how to search for solutions computationally has been an interesting problem. This paper proposes the first integer-programming-based formulation for the automatic composition of first species counterpoint, the simplest form of counterpoint, as a discrete optimization problem with constraints. Its difference from existing approaches such as constraint programming, stochastic method, and genetic algorithm is that it aims at efficiently accomplishing strict constraint satisfaction and optimality of the solution at the same time.

1. INTRODUCTION

Counterpoint is a theory for composing western polyphonic music. Especially, it has been developed through the vocal polyphony of the Renaissance period and then inherited in the music of later periods. It consists of many rules that regulate melodic and harmonic relationships within each voice or between voices.

Species counterpoint is a step-by-step approach to study counterpoint in the style of Palestrina, a well-known Renaissance composer, adopted in an influential book *Gradus ad Parnassum* of J.J.Fux [1]. There are five species, each of which has a fixed rhythmic structure. Every species has the same given melody called *cantus firmus* (fixed melody in Latin) whose rhythm is monotonous (one whole note in a measure), and students compose the counter-melodies that are compatible with the cantus firmus to complete polyphonic music (Figure 1). The rhythmic structures of the counter-melody of the respective species are as follows:

- First species: one whole note in a measure.
- Second species: two half notes in a measure
- Third species: four fourth notes in a measure.
- Fourth species: one note as *suspension* in an measure (two half notes are tied over consecutive measures)

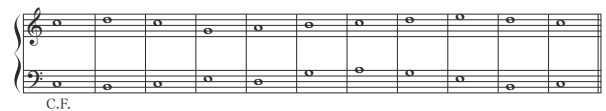


Figure 1. A realization of first species counterpoint from [2]. The lower voice is the cantus firmus.

- Fifth species: all of the previous species are mixed.

When studying counterpoint, composing a solution that does not have any violation of the rules is extremely difficult. Even checking whether a realization satisfies all of the rules is difficult for a student. Because of this, a student needs a human teacher who corrects their realizations. Even for professional teachers, satisfying all of the rules is not easy. Therefore, a computer system that can search for solutions (model answers) or modify the user's realizations would be helpful in the context of music education.

There have been different types of computational research for automatically finding solutions by computer. [3] is one of the earliest researches. It is based on a backtracking algorithm and could only provide solutions with some compromises because it uses a heuristic searching method. [4] is based on constraint programming and could provide exact solutions to small problems. [5] is based on a genetic algorithm in which the constraints are expressed by values that constitute the objective function. This approach has the merit that it can adjust the weights of respective constraints according to their importance. This allows it to find semi-optimal solutions quickly, although the optimal solutions are not assured. [6, 7] are based on a stochastic model to imitate the statistical tendency in existing pieces. They use dynamic programming to search for solutions efficiently, although they have a limitation in that they assume the Markov property that does not hold necessarily.

Following these models, this paper proposes a new computational model for composing first species counterpoint, the simplest form of species counterpoint, based on integer programming. The goal of this model is to accomplish “strict” constraint satisfaction and optimality of satisfying the “soft” and/or “global” constraints at the same time. Here, “soft” constraints are the rules that are less important than “strict” constraints that must be satisfied. Integer programming can formulate both strict and soft constraints. While the strict constraints can be formulated with mathematical equalities and inequalities, soft constraints can be formulated with objective function like in the model

of [5]. “Global” constraints mean constraints such as frequent uses of contrary motions (two melodies move in opposite directions) and conjunct motions (step-wise melodic motion, i.e., semitone and whole tone), which are about the global characteristics. These global constraints cannot be checked easily when assuming the Markov property like in [6, 7]. These can be treated with equalities, inequalities, or objective function in integer programming. The soft and/or global constraints are somewhat linked to the degree of goodness of composition to be optimized. We discuss it in Section 3.5.

Possible drawbacks of the proposed method are relatively large computational time and the complexity of programming many rules. Although it is beyond the scope of this paper to verify to which extent the integer-programming-based method is realistic to formulate more complex counterpoints, it is at least useful for the basic level of counterpoint education. For this purpose, this paper shows an integer-programming-based formulation of first species counterpoint and shows that a solution (a model answer) can be found in a realistic time.

2. INTEGER PROGRAMMING

Integer programming (IP) is a framework to solve discrete optimization problems whose variables are limited to integers. Famous examples of IP problems include the traveling-salesperson problem and the knapsack problem, which are NP-hard problems. Although IP is a powerful tool for finding optimal solutions for complex combinatorial problems, there is a significant applicability limitation. The constraints and objective function must be typically linear. Therefore, it is less flexible than constraint programming. In general, an IP problem can be expressed in the form as follows:

$$\begin{aligned} &\text{Maximize } z = \sum_j c_j x_j \\ &\text{subject to } \sum_j a_{ij} x_j \leq b_i \quad (i = 1, 2, \dots, m) \\ &\quad x_j = 0, 1, 2, \dots \quad (j = 1, 2, \dots, n), \end{aligned}$$

where a_{ij} , b_j , c_j are real numbers and x_j are integer variables. The linear inequalities in the middle, which are constraints, define a region surrounded by the hyperplanes. The integer points in this region are feasible solutions. Among these points, the optimal solutions are maximizers of the linear objective function z .

Once the problem is expressed in this form, efficient algorithms, such as the branch-and-bound and branch-and-cut algorithms, can find optimal solutions. Many solvers have been developed to solve IP problems. Recent solvers can deal with huge problems with thousands of variables thanks to the improvement of algorithms and computer performance. Commercial solvers includes Gurobi [8], CPLEX [9], Nuorium Optimizer [10], and free solvers include CBC [11] and GLPK [12]. In the experiment in this paper, CBC, which is the default solver that accompanies the python library PuLP [13], is used.

The aim of this paper is to show that counterpoint (at least first species counterpoint) can be formulated with IP. To do so, some formulation techniques, such as expressing logical operations by combining 0-1 binary variables and *big-M method*, are used as is discussed in Section 3. For further information about integer programming, refer to [14].

In the following section, the rules of first species counterpoint are enumerated, and they are translated into equalities, inequalities, and objective functions that are linear to show that a formulation of counterpoint is possible with IP.

The rules of counterpoint slightly differ from one textbook to another. This paper refers to the textbook of counterpoint [2], which is mainly based on a French textbook [15] but includes a survey on many existing counterpoint textbooks.

3. FORMULATION

3.1 Definition of Constants and Variables

3.1.1 Constants

Cantus firmus, a given melody, is denoted by an array of pitches, Cf . For example, we use the following melody in the experiment:

$$Cf = [0, -5, -7, -8, -7, -10, -5, -8, -10, -12].$$

$CfUp$ is an array that indicates whether the melodic direction of Cf from bar t to bar $t + 1$ is upward or not. For above Cf ,

$$CfUp = [0, 0, 0, 1, 0, 1, 0, 0, 0].$$

$CfDown$ is an array that indicates whether the melodic direction of Cf at bar t is downward or not. For above Cf ,

$$CfDown = [1, 1, 1, 0, 1, 0, 1, 1, 1].$$

T is the number of bars in the cantus firmus. For Cf ,

$$T = \text{length}(Cf).$$

The set of possible harmonic intervals, H , is defined as follows:

$$H = \{0, 3, 4, 5, 7, 8, 9, 12\}$$

The set of possible upward melodic intervals, M_+ , is defined as follows:

$$M_+ = \{1, 2, 3, 4, 5, 7, 8, 12\}.$$

The set of possible downward melodic intervals, M_- , is defined as follows:

$$M_- = \{-12, -8, -7, -5, -4, -3, -2, -1\}.$$

The set of possible melodic intervals, M , is the union of the two sets:

$$M = M_+ \cup M_-.$$

The set of possible melodic intervals that indicate leaps (melodic motions that are not conjunct), *smallLeaps*, is defined as follows:

$$\text{smallLeaps} = M - \{-2, -1, 1, 2\}.$$

The set of possible melodic intervals that indicate large leaps, *largeLeaps*, is defined as follows:

$$largeLeaps = M - \{-4, -3, -2, -1, 1, 2, 3, 4\}.$$

The set of bar numbers where *Cf* has a large leap is denoted by *CfLargeLeapT*.

The set of possible pitches in the register of soprano in a diatonic scale, *P*, is defined as follows:

$$P = \{0, 2, 4, 5, 7, 9, 11, 12, 14, 16, 17, 19, 21\}.$$

The set of time indices of size $T - n$, T_n , is defined as follows:

$$T_n = \{0, 1, 2, \dots, T - n - 1\} \quad (n \geq 0).$$

Minimum number of required contrary motions throughout the piece, *MinContrary*, is tentatively set as $T/2$:

$$MinContrary = T/2.$$

Minimum number of required conjunct motions throughout the piece, *MinConjunct*, is tentatively set as $T/2$:

$$MinConjunct = T/2.$$

3.1.2 Main Variables

0-1 binary variable that indicates whether the harmonic interval of counter-melody at bar t is i or not, $h_{t,i}$, is defined as follows:

$$h_{t,i} \in \{0, 1\} \quad (t \in T_0).$$

0-1 binary variable that indicates whether the melodic interval of counter-melody from bar t to bar $t + 1$ is i or not, $m_{t,i}$, is defined as follows:

$$m_{t,i} \in \{0, 1\} \quad (t \in T_1).$$

0-1 binary variable that indicates whether the pitch of counter-melody at bar t is u or not, $p_{t,u}$, is defined as follows:

$$p_{t,u} \in \{0, 1\} \quad (t \in T_0).$$

Integer variable that indicates harmonic interval between cantus firmus and counter-melody at bar t , *hInterval_t*, is defined as follows:

$$hInterval_t \in \mathbb{Z} \quad (t \in T_1).$$

Integer variable that indicates melodic interval of counter-melody from bar t to bar $t + 1$, *mInterval_t*, is defined as follows:

$$mInterval_t \in \mathbb{Z} \quad (t \in T_1).$$

3.1.3 Auxiliary Variables

0-1 binary variable that indicates whether the melodic direction of counter-melody is upward or not from bar t to bar $t + 1$, *up_t*, is defined as follows:

$$up_t \in \{0, 1\} \quad (t \in T_1).$$

0-1 binary variable that indicates whether the melodic direction of counter-melody is downward or not, *down_t*, is defined as follows:

$$down_t \in \{0, 1\} \quad (t \in T_1).$$

0-1 binary variable that indicates whether the melodic motion of counter-melody is conjunct or not from bar t to bar $t + 1$, *conjunct_t*, is defined as:

$$conjunct_t \in \{0, 1\} \quad (t \in T_1).$$

0-1 binary variable that indicates whether the melodic motion between cantus firmus and counter-melody is contrary or not from bar t to bar $t + 1$, *contrary_t*, is defined as:

$$contrary_t \in \{0, 1\} \quad (t \in T_1).$$

0-1 binary variable that indicates whether the melodic motion of counter-melody from bar t to bar $t + 2$ changes direction at bar $t + 1$ or not, *turn_t*, is defined as follows:

$$turn_t \in \{0, 1\} \quad (t \in T_1).$$

Integer variable that indicates the sum of the numbers of leaps of counter-melody at bars t and $t + 1$ (to know whether there is a consecutive leaps or not), *consecutiveLeap_t*, is defined as follows:

$$consecutiveLeap_t \in \{0, 1, 2\} \quad (t \in T_2).$$

0-1 binary variable that indicates whether the melodic motion in counter-melody from bar t is a large leap or not, *largeLeap_t*, is defined as follows:

$$largeLeap_t \in \{0, 1\} \quad (t \in T_1).$$

Integer variable that indicates the maximum value of pitches in the whole counter-melody, *maxP*, is defined as follows:

$$maxP \in \{x \in \mathbb{N} \mid 0 \leq x \leq 21\}.$$

3.2 Basic relationship between variables

There must be constraints between variables defined above. This subsection presents the basic relationship between m , p , and h . Only one of the possible intervals or pitches is realized at bar t . These constraints are expressed by the following equations:

$$\sum_{i \in M} m_{t,i} = 1 \quad (t \in T_1), \quad (1)$$

$$\sum_{u \in P} p_{t,u} = 1 \quad (t \in T_0), \quad (2)$$

$$\sum_{i \in H} h_{t,i} = 1 \quad (t \in T_0). \quad (3)$$

Harmonic and melodic intervals at bar t are expressed by h and m , respectively:

$$hInterval_t = \sum_{i \in H} i \cdot h_{t,i} \quad (t \in T_0), \quad (4)$$

$$mInterval_t = \sum_{i \in M} i \cdot m_{t,i} \quad (t \in T_1). \quad (5)$$

There must be a consistency between melodic interval and harmonic interval:

$$mInterval_t = (hInterval_{t+1} + Cf[t+1]) - (hInterval_t + Cf[t]) \quad (t \in T_1). \quad (6)$$

There must be a consistency between two different expressions of pitch in the counter-melody (when $p_{t,u} = 1$, $u \cdot p_{t,u}$ is the pitch at bar t and it must equals to $Cf[t] + hInterval_t$). Therefore,

$$\sum_{u \in P} u \cdot p_{t,u} = Cf[t] + hInterval_t \quad (t \in T_0). \quad (7)$$

3.3 Preparation of Auxiliary Variables

This subsection presents the constraints for preparing other auxiliary variables.

If one of the melodic interval in M_+ occurs at bar t , the melodic direction at t must be upward, and vice versa for the downward direction. The direction of melody is either up or down (repetition of the same pitches is excluded in advance by M). Therefore,

$$up_t \geq m_{t,i} \quad (t \in T_1, i \in M_+), \quad (8)$$

$$down_t \geq m_{t,i} \quad (t \in T_1, i \in M_-), \quad (9)$$

$$up_t + down_t = 1 \quad (t \in T_1). \quad (10)$$

When both cantus firmus and counter-melody have the same melodic direction, there must be a contrary motion at bar t ($contrary_t = 1$). Therefore,

$$contrary_t = CfUp[t] \cdot down_t + CfDown[t] \cdot up_t \quad (t \in T_1). \quad (11)$$

When there is a conjunct motion at bar t of counter-melody, $conjunct_t$ must become 1. Therefore,

$$conjunct_t = m_{t,1} + m_{t,-1} + m_{t,2} + m_{t,-2} \quad (t \in T_1). \quad (12)$$

$consecutiveLeap_t$ is the number of consecutive (small) leaps at bar t and $t+1$ in counter-melody. Therefore,

$$consecutiveLeap_t = \sum_{i \in smallLeaps} (m_{t,i} + m_{t+1,i}) \quad (t \in T_2). \quad (13)$$

$largeLeap_t$ indicates whether there is a large leap or not at bar t in counter-melody. Therefore,

$$largeLeap_t = \sum_{i \in LargeLeaps} m_{t,i} \quad (t \in CfLargeLeapT). \quad (14)$$

There must be consistency between variables up , $down$, and $turn$ (for example, if $up_t = 1$ and $down_{t+1} = 1$, $turn_t$ must be 1. And so on.). Therefore,

$$up_t + down_{t+1} \leq 1 + turn_t \quad (t \in T_1), \quad (15)$$

$$down_t + up_{t+1} \leq 1 + turn_t \quad (t \in T_1), \quad (16)$$

$$up_t + up_{t+1} \leq 1 + (1 - turn_t) \quad (t \in T_1), \quad (17)$$

$$down_t + down_{t+1} \leq 1 + (1 - turn_t) \quad (t \in T_1). \quad (18)$$

3.4 Rules of Counterpoint

3.4.1 Limited use of harmonic interval 0.

Harmonic interval 0 is possible only at the first bar ($t = 0$) and the last bar ($t = T - 1$). Therefore,

$$h_{t,0} = 0 \quad (t \in 1, 2, 3, \dots, T-2). \quad (19)$$

3.4.2 Prohibitions of parallel fifths and octaves

Consecutive occurrence of harmonic intervals of fifth and that of eighth are prohibited. Therefore,

$$h_{t,i_1} + h_{t+1,i_2} \leq 1 \quad (t \in T_1, i_1, i_2 \in \{0, 12, 24\}), \quad (20)$$

$$h_{t,i_1} + h_{t+1,i_2} \leq 1 \quad (t \in T_1, i_1, i_2 \in \{7, 19\}). \quad (21)$$

3.4.3 Prohibitions of hidden fifths and octaves

Harmonic motions that reach fifth or eighth in a parallel motion are prohibited. (For example, when both voices go up, $up_t = 1$ and this must imply $h_{t+1,24} = 0$, which means harmonic interval 24 does not occur.) Therefore,

$$\text{if } CfUp[t] == 1 \quad (t \in T_1),$$

$$h_{t+1,24} \leq 1 - up_t, \quad (22)$$

$$h_{t+1,19} \leq 1 - up_t, \quad (23)$$

$$h_{t+1,12} \leq 1 - up_t, \quad (24)$$

$$h_{t+1,7} \leq 1 - up_t, \quad (25)$$

$$h_{t+1,0} \leq 1 - up_t. \quad (26)$$

$$\text{else if } CfDown[t] == 1,$$

$$h_{t+1,24} \leq 1 - down_t, \quad (27)$$

$$h_{t+1,19} \leq 1 - down_t, \quad (28)$$

$$h_{t+1,12} \leq 1 - down_t, \quad (29)$$

$$h_{t+1,7} \leq 1 - down_t, \quad (30)$$

$$h_{t+1,0} \leq 1 - down_t. \quad (31)$$

3.4.4 Prohibited arpeggio patterns of triads

In consecutive three notes in counter-melody, arpeggio of triads (major, minor, diminished, and augmented) in one direction (without turn) must not appear. Therefore,

$$m_{t,3} + m_{t,4} + m_{t+1,3} + m_{t+1,4} \leq 1 \quad (t \in T_2), \quad (32)$$

$$m_{t,-3} + m_{t,-4} + m_{t+1,-3} + m_{t+1,-4} \leq 1 \quad (t \in T_2), \quad (33)$$

$$m_{t,3} + m_{t,4} + m_{t+1,5} \leq 1 \quad (t \in T_2), \quad (34)$$

$$m_{t,-5} + m_{t+1,-3} + m_{t+1,-4} \leq 1 \quad (t \in T_2), \quad (35)$$

$$m_{t,5} + m_{t+1,3} + m_{t+1,4} \leq 1 \quad (t \in T_2), \quad (36)$$

$$m_{t,-3} + m_{t,-4} + m_{t+1,-5} \leq 1 \quad (t \in T_2). \quad (37)$$

3.4.5 Prohibition of two consecutive leaps in the same direction

If there are consecutive leaps ($consecutiveLeap_t = 2$), they must form a turn ($turn_t = 1$). And if $turn_t = 0$, $consecutiveLeap_t$ must be less than 2 (i.e., no consecutive leaps at t). Therefore,

$$consecutiveLeap_t \leq 1 + turn_t \quad (t \in T_2). \quad (38)$$

3.4.6 Prohibition of three-times repetitions of the same pitch in five consecutive notes

There must not be three same pitches in five consecutive notes (repetition of the same pitch in consecutive two notes has been already excluded because M does not include 0.). Therefore,

$$\sum_{s=t}^{t+4} p_{s,u} \leq 2 \quad (t \in T_4, u \in P). \quad (39)$$

3.4.7 Prohibition of simultaneous large leaps between two voices

When $t \in CfLargeLeapT$, if there is a large leap at t ($largeLeap_t = 1$), there must not be a parallel motion at t ($contrary_t$ must be 1) to avoid simultaneous parallel leaps between two voices. And if there is a parallel motion at t ($contrary_t = 0$), there must not be a large leap at t in counter-melody ($largeLeap_t = 0$) to avoid simultaneous large leaps. Therefore,

$$largeLeap_t \leq contrary_t \quad (t \in CfLargeLeapT). \quad (40)$$

3.4.8 Large leaps must be preceded or succeeded by opposite melodic movement

This rule is unnecessary because the rule 3.4.5 is sufficient.

3.4.9 In two-voice counterpoint, it is desirable to avoided voice crossing

In this paper, this rule is unnecessary because voice crossing is avoided by the definition of H .

3.4.10 Consecutive parallel movement of third or sixth is limited to three times

The number of consecutive parallel movements of the same harmonic intervals concerning third and sixth must be at most 3. Therefore,

$$\sum_{s=t}^{t+3} (h_{s,3} + h_{s,4}) \leq 3 \quad (t \in T_3), \quad (41)$$

$$\sum_{s=t}^{t+3} (h_{s,8} + h_{s,9}) \leq 3 \quad (t \in T_3), \quad (42)$$

$$\sum_{s=t}^{t+3} (h_{s,15} + h_{s,16}) \leq 3 \quad (t \in T_3), \quad (43)$$

$$\sum_{s=t}^{t+3} (h_{s,20} + h_{s,21}) \leq 3 \quad (t \in T_3). \quad (44)$$

3.4.11 Rules for desired melody

According to the textbook we refer to, the criteria for desired melody are as follows:

1. Contrary motions should be used as much as possible.
2. Long sequences of conjunct motion are included.
3. The melody has a "will" as a whole.

4. The voice had better stay within the adequate register of the part.

One way to interpret the first and second points is to regard them as maximizations of the number of contrary motions and that of conjunct motions, which is easy to implement as an objective function described in the next section. Another possible interpretation, which is compatible with the first one, is to introduce minimum thresholds for the number of contrary motions ($MinContrary$) and that of conjunct motions ($MinConjunct$). They can be implemented as the following constraints:

$$\sum_{t \in T_1} contrary_t \geq MinContrary, \quad (45)$$

$$\sum_{t \in T_1} conjunct_t \geq MinConjunct. \quad (46)$$

The fourth point can be satisfied by setting P , possible pitches for counter-melody. The third point is the most difficult to understand. One way to interpret it is to introduce a climax (the highest pitch) at unique t , and to suppress the number of zigzag movements (turns) that look like "hesitating." The latter can be implemented as a minimization of the number of turns, which is described in the next section. The former can be implemented by the following constraints:

$$\sum_{t \in T_0} climax_t = 1, \quad (47)$$

$$(Cf[t] + hInterval_t) + (1 - climax_t) \leq maxP \quad (t \in T_0), \quad (48)$$

$$maxP \leq (Cf[t] + hInterval_t) + Width \cdot (1 - climax_t) \quad (t \in T_0). \quad (49)$$

Here the last inequality uses the technique of Big-M method that can switch the inequality. If $climax_t = 1$, the term of $Width$ (this constant is sufficiently "big") is eliminated and the pitch at bar t , $Cf[t] + hInterval_t$, coincides with $maxP$. Therefore, the climax is realized at bar t . If $climax_t = 0$, the inequality does not work as a constraint because $Width$ is sufficiently large.

3.5 Objective Function

As we discussed the rule 3.4.11, we can consider that there are three elements that influence the goodness, namely, the numbers of contrary motions, conjunct motions, and turns. Therefore, a way to define the criteria as an objective function is to add or subtract these numbers as follows:

Minimize

$$\sum_{t \in T_2} turn_t - \sum_{t \in T_2} contrary_t - \sum_{t \in T_2} conjunct_t.$$

Of course, we can consider introducing coefficients for these three criteria. Also, there may be other possible criteria. The best choice of the coefficients and the criteria may differ from one user to another. When this proposed



Figure 2. A generated result. The lower voice is cantus firmus, a given melody used in the textbook [2].

criterion	occurrence
turn	6
contrary motions	6
conjunct motions	6

Table 1. The values of the three criteria of the objective function for the piece in Figure 2

method is used in an actual application, some choices may be provided to the users so that they can find their preferences for the objective function. In any case, we assume that the possible criteria can be expressed as linear functions by setting adequate coefficients on them. Therefore, we adopt the above objective function tentatively for simplicity of discussion without loss of generality.

Therefore, integer programming can be applied to this problem because of this assumption and the linearity of the constraints. The constraints define possible solutions that satisfy the strict rules of counterpoint, and the object function defines the optimality within them from the chosen criteria for the goodness of composing counter-melody.

3.6 Generated Result

The formulation of the previous section was implemented with PuLP, a python library that can model linear optimization problems. CBC, the free default solver of PuLP was used to search for the solution. The type of computer was MacBook Pro 2016 with 2.6 GHz quad-core Intel Core i7.

Figure 2 shows an optimal solution found. It took 3.88 seconds to search for it. The value of the objective function is -6 . The contribution of each criterion is shown in Table 1. From this table, we can see that three criteria are equally satisfied in a good balance. The climax of the counter-melody is realized at bar 2. Thanks to the rule of climax in 3.4.11, the melody has a certain degree of directionality as a melody (hopefully a "will" described in 3.4.11). The melody goes down and then goes up in the last part. Thanks to the modest number of turns and relatively many contrary and conjunct motions, the counter-melody looks stable and independent from the cantus firmus. For comparison, another generated result is shown in Figure 3. In this case, only the number of turns was maximized to observe the importance of the three criteria. However, because of the large number of turns and less conjunct motions, the counter-melody looks relentless, and its unity as a melody looks a little broken. The original objective function seems to work well from this comparison.



Figure 3. A piece generated with a different objective function. Only the number of turns was taken into account as the objective function (maximized). In this piece, the number of turns, contrary motions, and conjunct motions are 8, 6, 5, respectively.

4. CONCLUSIONS

This paper presented a new formulation of first species counterpoint based on integer programming. The rules of counterpoint were expressed by linear constraints and objective function so that integer programming could be applied. As a result, optimal solutions were found within a realistic time. There are remaining questions. Although counterpoint consists of explicit rules in the textbook, are they enough to create excellent pieces? Are the generated pieces natural enough to the extent that listeners can not distinguish whether they are composed by humans or by a computer? To answer these questions, an evaluation of generated pieces and a comparison between the generated pieces and human composers' pieces would be necessary. Also, formulating counterpoints of more complex species and larger numbers of voices is a future task.

Acknowledgments

This work was supported by JSPS KAKENHI Grant Number 19K13024.

5. REFERENCES

- [1] J. J. Fux, *The study of counterpoint from Johann Joseph Fux's Gradus ad Parnassum*. WW Norton & Company, 1965.
- [2] Y. Hiroshi, *Traité de contrepoint d'après les méthodes du Conservatoire de Paris*. Ongakunotomosha, 2012.
- [3] W. Shottstaedt, "Automatic counterpoint," in *Current directions in computer music research*, 1989.
- [4] A. Torsten, C. Anagnostopoulou, and M. Alcorn, "Strasheela: design and usage of a music composition environment based on the Oz programming model," in *Proceedings of International Conference on Multiparadigm Programming in Mozart/Oz*, Charleroi, 2005, pp. 277–291.
- [5] D. Herremans and K. Sørensen, "Composing first species counterpoint with a variable neighbourhood search algorithm," in *Journal of Mathematics and the Arts* 6.4, 2012, pp. 169–189.
- [6] M. Farbood and B. Schöner, "Analysis and synthesis of Palestrina-style counterpoint using Markov chains," in *Proceedings of International Computer Music Conference*, 2001.

- [7] T. Tanaka and et al., “Automatic music composition based on counterpoint and imitation using stochastic models,” in *Proceedings of 7th Sound and Music Computing Conference*, Barcelona, 2010, pp. 213–218.
- [8] <https://www.gurobi.com/>.
- [9] <https://www.ibm.com/products/ilog-cplex-optimization-studio>.
- [10] <https://www.msi.co.jp/nuopt/index.html>.
- [11] <https://github.com/coin-or/Cbc>.
- [12] <https://www.gnu.org/software/glpk/>.
- [13] <https://coin-or.github.io/pulp/index.html>.
- [14] D. S. Chen, R. Batson, and Y. Dang, *Applied Integer Programming: Modeling and Solution*. Wiley, 2010.
- [15] G. Noël and B. Marcel, *Traité de contrepoint*. Durand, 1964.